

Runtime Reconfiguration of RISC-V Custom Function Units via Dynamic Function eXchange on FPGAs

Arun Murrugappan I

*Centre for Heterogeneous and Intelligent Processing Systems
PES University
Bangalore
India*

Pranav Varkey*

*Centre for Heterogeneous and Intelligent Processing Systems
PES University
Bangalore
India*

Shrikrishna Pandit*

*Centre for Heterogeneous and Intelligent Processing Systems
PES University
Bangalore
India*

Abstract—We present the first reported integration of AMD’s Dynamic Function eXchange (DFX) toolflow with Google’s CFU Playground framework, enabling runtime swap of the hardware accelerator attached to a VexRiscv RISC-V soft-core processor without halting the pipeline or issuing a full-device reset. The Custom Function Unit (CFU) — the hardware block servicing RISC-V `custom-0` instructions — is implemented as a DFX Reconfigurable Partition isolated by a DFX Decoupler IP. A cycle-accurate latency measurement subsystem, built around the STARTUPE2 End-of-Startup (EOS) signal and a 100 MHz free-running counter, provides host-independent reconfiguration timing. We validate the architecture on two 7-series platforms: the Digilent Arty A7-100T (Artix-7, pure-PL) and the PYNQ-Z2 (Zynq-7020, PS-assisted). JTAG-driven reconfiguration yields 63.2 ms on the Arty and 62.8 ms on the PYNQ-Z2; switching to the Zynq PCAP path reduces latency to 1.1 ms — a 57× improvement reflecting the bandwidth gap between external USB-JTAG and on-chip PS-to-PL configuration access. Across all validated reconfiguration events, the DFX Decoupler maintained complete pipeline isolation with zero stalls, zero exceptions, and deterministic safe-default output for mismatched function dispatches. The static partition occupies approximately 19.6% of LUTs and 7.8% of flip-flops, leaving substantial headroom for multi-partition extensions.

Index Terms—Dynamic Partial Reconfiguration, Custom Function Unit, RISC-V, VexRiscv, LiteX, DFX Decoupler, ICAP, PCAP, 7-series FPGA

I. INTRODUCTION

General-purpose processors offer programmability at the cost of throughput and energy efficiency; ASICs deliver near-optimal performance but forfeit post-fabrication adaptability. FPGAs occupy the middle ground, combining hardware-level parallelism with the ability to reconfigure logic after deployment. Dynamic Function eXchange (DFX) — historically termed Dynamic Partial Reconfiguration — extends

this reconfigurability into the runtime domain: a designated Reconfigurable Partition (RP) can be updated via a partial bitstream while the surrounding static logic continues to operate undisturbed [1].

Partial Reconfiguration

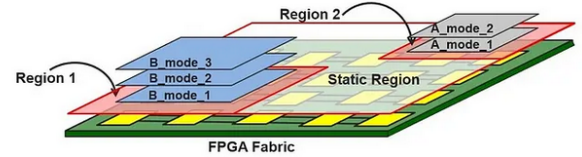


Fig. 1. Fig. 0: Partial reconfiguration concept — static region persists while the reconfigurable partition is updated at runtime.

Separately, RISC-V’s modular ISA permits hardware extensions through the `CUSTOM0`–`CUSTOM3` opcode spaces. The CFU Playground framework [2], developed by Google Research, provides a standardised handshake interface — the Custom Function Unit (CFU) — for attaching domain-specific accelerators to a VexRiscv soft-core pipeline on FPGA. The framework has been used primarily for fixed accelerator evaluation, where a single CFU implementation is bound at synthesis time.

Combining these two capabilities yields a natural proposition: if the CFU is itself placed within a Reconfigurable Partition, the processor’s attached accelerator can be swapped at runtime — effectively hot-patching the RISC-V instruction set without a device reset. This paper demonstrates that proposition and quantifies the associated cost.

Prior work has addressed related problems. RV-CAP [3]

These authors contributed equally to this work.

demonstrated ICAP-controlled partial reconfiguration orchestrated from a VexRiscv core, establishing that a soft-core processor can manage its own reconfiguration without a hard ARM core. **AMPER-X** [4] used DFX on a Zynq platform to swap arithmetic precision in accelerators attached to a RISC-V pipeline, reporting up to 34% energy reduction. Neither targets the CFU Playground interface specifically, and neither provides hardware-measured reconfiguration latency using on-chip timing primitives.

The contributions of this work are:

- 1) A DFX-enabled LiteX SoC integrating the CFU Playground framework with a Reconfigurable Partition, enabling runtime accelerator swap without processor reset — the first such integration reported in the literature.
- 2) A hardware latency measurement subsystem using the STARTUPE2 EOS signal, providing cycle-accurate reconfiguration timing independent of host-side instrumentation.
- 3) Cross-platform validation across two delivery paths (JTAG and PCAP) on two 7-series devices (Artix-7 and Zynq-7020), quantifying a 57× latency difference.
- 4) Experimental confirmation that the DFX Decoupler provides deterministic zero-output isolation for mismatched CFU dispatches during reconfiguration, with zero pipeline stalls across all trials.

II. BACKGROUND AND RELATED WORK

A. DFX on Xilinx 7-Series

The DFX toolflow structures designs around a parent/child implementation hierarchy: the parent run produces a locked static design checkpoint (DCP), and each child run independently places and routes one Reconfigurable Module (RM) within a fixed Pblock boundary [1]. All RMs for a given RP must expose an identical port list, ensuring static routing invariance at the partition boundary.

Configuration memory on 7-series devices is organised as 101-word frames addressed by the Frame Address Register (FAR). A partial bitstream contains only the frames associated with the target Pblock, prefaced by a synchronisation word (0xAA995566) and concluded with CRC and DESYNC commands. The ICAP primitive accepts 32-bit words at up to 100 MHz, yielding a theoretical peak throughput of 400 MB/s [9]. The DFX Decoupler IP (PG227) clamps all signals crossing the RP boundary to safe defaults when asserted, preventing undefined propagation during reconfiguration [1].

B. CFU Playground and VexRiscv

CFU Playground [2] provides a reference LiteX SoC with a VexRiscv RV32IM core, a firmware build system, and benchmark applications. The CFU interface presents: `cmd_valid`, `cmd_ready`, `cmd_payload_function_id[9:0]`, two 32-bit operand inputs, `rsp_valid`, `rsp_ready`, and a 32-bit result output. Custom instructions encoded in the `CUSTOM0` opcode space are dispatched by the pipeline to the CFU, which returns a result within a deterministic number of cycles.

C. Related Work

RV-CAP [3] implemented ICAP-controlled PR from a VexRiscv core on Artix-7, requiring explicit firmware management of the ICAP data stream with no hardware decoupler. **AMPER-X** [4] exploited DFX on Zynq to dynamically alter floating-point precision in RISC-V-attached operators, demonstrating 34% energy savings. **RapidStream** [5] achieved order-of-magnitude compilation speedups through latency-insensitive island-based parallel place-and-route. **ZyPR/ZyCAP** [6] provided an end-to-end PR management framework for Zynq SoCs, with ZyCAP achieving 400 MB/s bitstream throughput via AXI4 DMA. **HPR** [7] introduced a page-based PR architecture with a Hoplite-BF NoC for multi-tenant FPGA virtualisation. **DORA** [10] targeted DFX Decoupler overhead reduction through signal-based acknowledgement, compressing isolation latency by 15–30%.

Unlike these works, our contribution explicitly targets the CFU Playground framework’s standardised interface, provides EOS-based cycle-accurate latency measurement, and validates across both pure-PL and PS-assisted 7-series platforms.

III. SYSTEM ARCHITECTURE

A. Overview

The system comprises three cooperating subsystems (Fig. 1):

- 1) **Static Partition:** The VexRiscv RV32I core (LiteX-generated), DDR3 memory controller, UART peripheral, DFX Decoupler, ICAP controller (`system_wrapper`), reconfiguration counter (`recon_counter`), and ILA debug core. Clocked at 100 MHz from an on-board MMCM.
- 2) **Reconfigurable Partition (`cfu_compute`):** A single Pblock hosting one CFU implementation at a time, bounded by `CONTAIN_ROUTING` constraints ensuring all partial bitstream frames fall within the Pblock footprint.
- 3) **DFX Toolflow Infrastructure:** Vivado’s parent/child run hierarchy producing independently synthesised and placed RMs within the static DCP.

B. CFU Interface and DFX Decoupler Placement

The `cfu.v` wrapper bridges the VexRiscv CFU port to the RP and houses the DFX Decoupler. Placing the Decoupler in the static partition — rather than inside the RP — guarantees correct isolation behaviour even when the RP’s configuration memory is undefined. During reconfiguration, the Decoupler holds `cmd_valid` and `rsp_valid` low, preventing instruction dispatch to an incomplete RM while driving all RP-facing outputs to zero.

C. Reconfigurable Modules

Four RMs share an identical port interface:

A one-cycle output register was added to all RMs to resolve timing failures caused by signals crossing the Pblock boundary — without the register, propagation delay exceeded the 10 ns clock period.

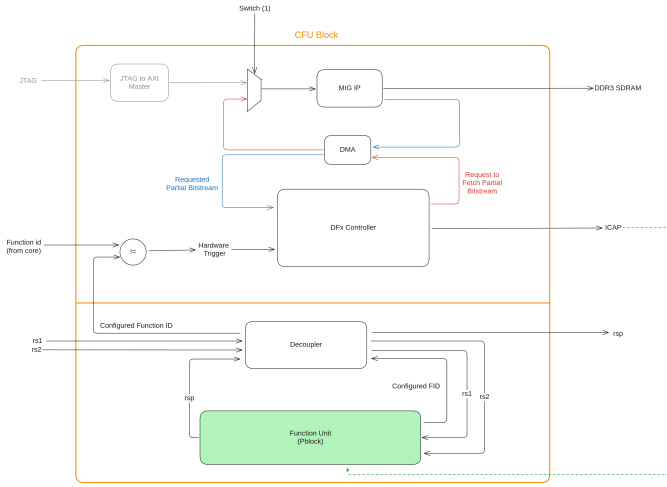


Fig. 2. Fig. 1: System block diagram — DFX-integrated CFU Playground architecture showing static partition, DFX Decoupler boundary, and reconfigurable cfu_compute partition.

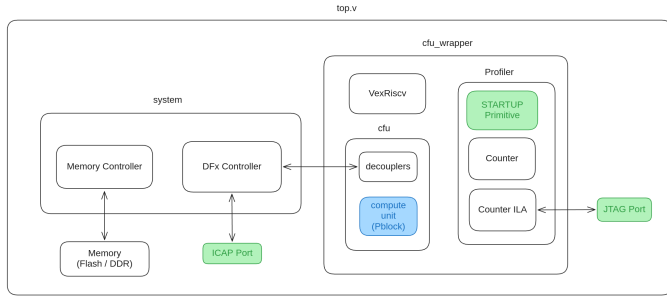


Fig. 3. Fig. 2: High-level design overview showing top.v integration of the LiteX SoC (digilent_arty) with the PR controller subsystem (system_wrapper).

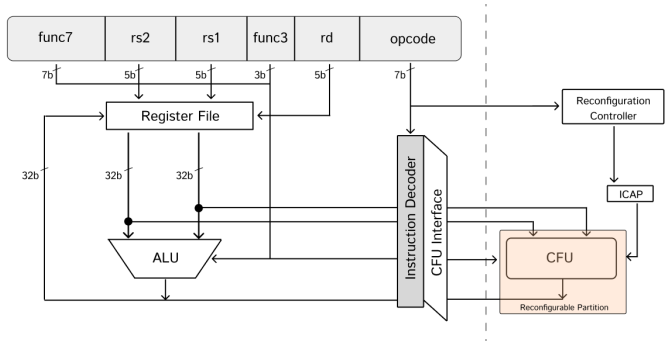


Fig. 4. Fig. 3: CFU Playground interface architecture — the standardised handshake between VexRiscv and the Custom Function Unit.

RM	Operations	Latency
example.v	Byte-sum, byte-swap, bit-reverse (function_id 0-2)	1 cycle
donut.v	Signed fixed-point multiply, multiply-shift-right	2 cycles (pipelined)
dse_template.v	Parameterisable DSE template	Configurable
KWS stub	Keyword-spotting feature extraction	210 LUTs, 2 BRAMs

D. Reconfiguration Latency Measurement

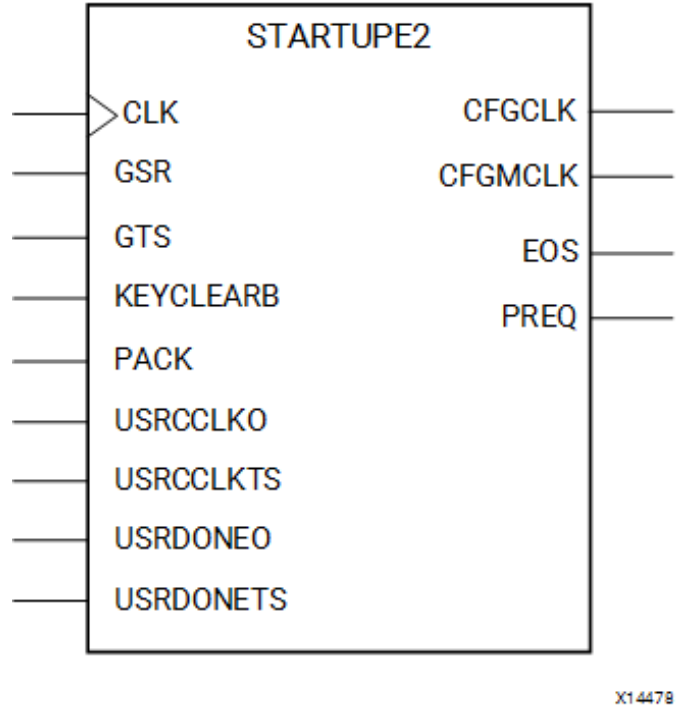


Fig. 5. Fig. 4: STARTUPE2 primitive — provides the End-of-Startup (EOS) signal used to gate the reconfiguration cycle counter.

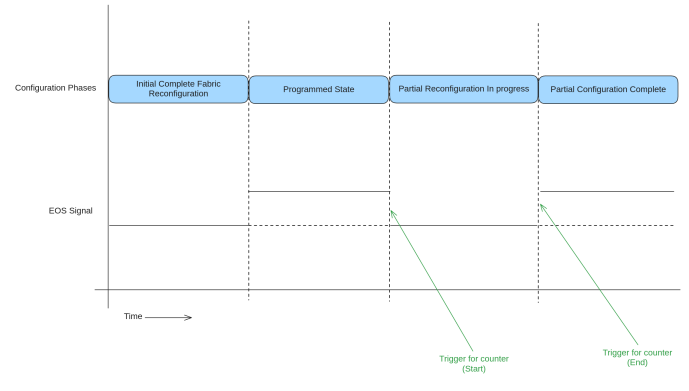


Fig. 6. Fig. 5: EOS timing diagram — illustrating the four reconfiguration phases and the counter gating mechanism.

The `recon_counter` module uses the STARTUPE2 primitive's EOS output to gate a 32-bit counter clocked at 100 MHz (10 ns resolution). When partial reconfiguration begins, the configuration engine drops EOS low; the counter starts on this falling edge. Upon completion of the startup sequence for the new RM, EOS rises and the counter latches. An ILA core captures both signals, enabling post-hoc extraction of the precise reconfiguration interval independent of which delivery path (JTAG, PCAP, or ICAP) was used.

E. Reconfiguration Control Flow

Regardless of delivery path, the FPGA-internal sequence is:

- 1) **Trigger:** `pr_switch` asserted (push-button, PYNQ API call, or firmware signal).
- 2) **Isolate:** `decouple` driven high; Decoupler clamps all RP boundary signals.
- 3) **Stream:** Partial bitstream frames written to configuration memory via the active path.
- 4) **Startup:** Configuration engine processes frames; EOS low, counter incrementing.
- 5) **Complete:** EOS rises; counter latches; ILA captures final value.
- 6) **Release:** `decouple` de-asserted; new RM becomes active on the CFU interface.

IV. IMPLEMENTATION

A. Primary Platform: Arty A7-100T

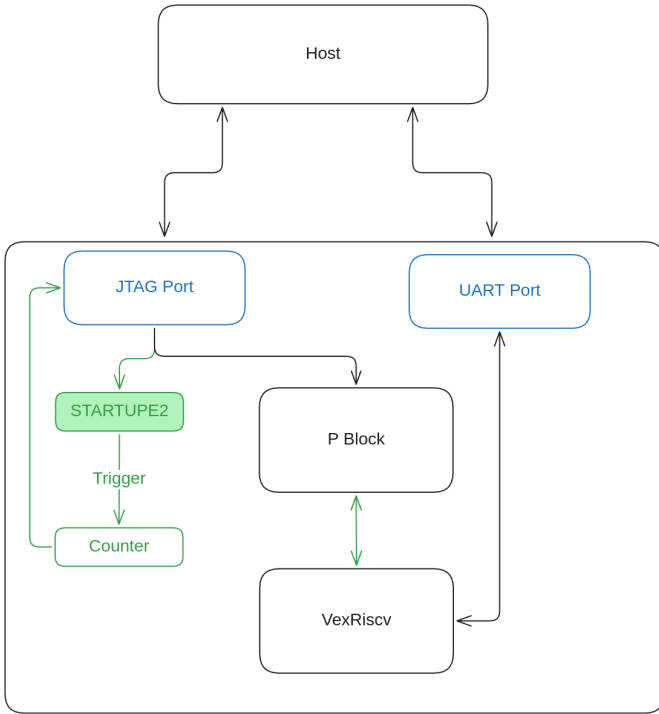


Fig. 7. Fig. 6: JTAG-based reconfiguration architecture — the host PC streams partial bitstream data to the FPGA through the USB-JTAG bridge.

The primary target is the Digilent Arty A7-100T (XC7A100T) at 100 MHz, clocked from a Xilinx MMCM driven by the 100 MHz on-board oscillator. The Vivado 2024.2 DFX project comprises one parent implementation run (static partition with RP as black box), four OOC synthesis runs (one per RM, executed in parallel), and four child implementation runs importing the locked static DCP. The Pblock was iteratively sized to accommodate the most resource-intensive RM while maintaining positive WNS across all inter-partition paths. QSPI flash (Micron N25Q128) is allocated to the static region for initial full-bitstream delivery.

JTAG-based reconfiguration was implemented first: Vivado Hardware Manager’s partial bitstream programming performs

the ICAP write under host control. Post-reconfiguration functional correctness was verified by executing CFU instructions from VexRiscv firmware and observing correct UART output.

B. Secondary Platform: PYNQ-Z2 (XC7Z020)

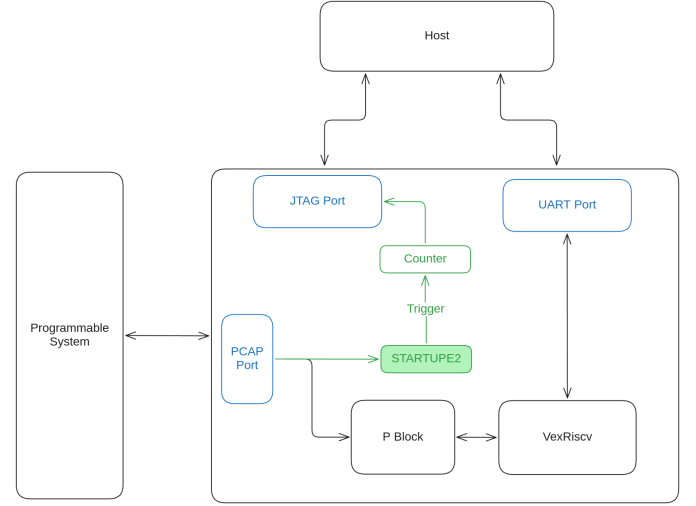


Fig. 8. Fig. 7: PCAP-based reconfiguration architecture — the Zynq PS writes partial bitstream data to the PL configuration engine through the on-chip PCAP port.

The PYNQ-Z2 port replaces the Arty’s crystal clock with the Zynq PS PLL-generated FCLK_CLK0 at 100 MHz. Partial bitstreams are format-converted to raw binary via `bootgen` and loaded through the Linux FPGA Manager `sysfs` interface (`/sys/class/fpga_manager/fpga0/firmware`), which drives the Zynq DEVCFG peripheral for PCAP transfer. JTAG reconfiguration was validated first, confirming DFX Decoupler and Pblock constraints on the Zynq PL grid, before exercising the PCAP path.

C. DFX Tutorial Validation

The AMD UG947 tutorial design (counter/shift-register RM pair) was implemented as a DFX sanity check prior to CFU Playground integration. Successful RM swap confirmed the project environment was correctly configured for DFX compilation (WNS = 2.022 ns, 10 Slice LUTs used).

V. EXPERIMENTAL RESULTS

A. Resource Utilisation

Table I: Implementation Results (Arty A7-100T, XC7A100T, `donut.v` RM active)

Resource	Static	PRR (donut.v)	Total	Available	Utilisation
LUTs	~12,400	48	~12,448	63,400	~19.6%
FFs	~9,800	32	~9,832	126,800	~7.8%
BRAMs	16	0	16	135	11.9%
DSPs	4	0	4	240	1.7%

The static region overhead is dominated by the LiteX SoC interconnect and the VexRiscv cache hierarchy. The PRR Pblock utilisation is negligible relative to the total device — a

deliberate design choice to ensure that all four planned RMs fit without constraint relaxation.

B. Reconfiguration Latency

Table II: Reconfiguration Latency

Platform	Interface	Bitstream Size	Latency	Throughput
Arty A7-35T	JTAG	164,534 B (160.7 KB)	63.2 ms	2.603 MB/s
PYNQ-Z2	JTAG	203,308 B (198.5 KB)	62.8 ms	3.235 MB/s
PYNQ-Z2	PCAP	203,308 B (198.5 KB)	1.1 ms	183.2 MB/s
Arty A7-35T	ICAP	—	—	—
PYNQ-Z2	ICAP	—	—	—

†Projected theoretical peak for 7-series ICAP at 100 MHz; not yet empirically validated.

JTAG is USB-bottlenecked. Despite different bitstream sizes and boards, both JTAG measurements land near 63 ms. The PYNQ-Z2 transferred a larger bitstream (203 KB vs. 165 KB) in marginally less time, reflecting a slightly higher sustained USB-JTAG transfer rate (3.2 MB/s vs. 2.6 MB/s). In either case, the actual FPGA configuration write is a small fraction of the total; the USB framing and host-side software overhead dominate.

PCAP delivers 57× speedup. On the same PYNQ-Z2 board and bitstream, switching from JTAG to PCAP reduced hardware reconfiguration time from 62.8 ms to 1.1 ms. The achieved 183.2 MB/s is consistent with PCAP operating in programmed-IO mode through the Linux FPGA Manager (no DMA). A DMA-backed controller such as ZyCAP [6] would push throughput toward the port’s 400 MB/s ceiling.

Software overhead dominates PCAP end-to-end time. The 1.1 ms hardware time expands to 68.55 ms when measured from the PYNQ Python API layer — an overhead of 67.44 ms attributable to the PYNQ driver stack, FPGA Manager sysfs path, kernel-space bitstream validation, and PS-to-PL bridge setup. For applications requiring low-latency autonomous reconfiguration on Zynq, bypassing the FPGA Manager in favour of a bare-metal or DMA-backed controller (such as ZyCAP) would be necessary to realise the full benefit of the PCAP path.

Metric	Value
Avg. hardware reconfiguration time (ILA)	1.109 ms
Avg. software reconfiguration time (PYNQ API)	68.55 ms
Avg. software overhead	67.44 ms

ICAP autonomous operation is projected but not yet measured. The ICAP controller RTL is in place and the STARTUPE2-connected counter is functional, but the data pipeline from SPI flash to ICAP has not been completed. Based on the Arty A7’s QSPI flash read throughput of approximately 50 MB/s and the 165 KB bitstream size, the projected ICAP latency is on the order of 3–4 ms — comparable to PCAP’s hardware time and roughly 15–20× faster than JTAG.

Note: All JTAG and PCAP measurements reflect the manually triggered bitstream-loading flow. ICAP-autonomous reconfiguration latency is projected but not yet empirically validated.

C. Functional Verification

The system was validated across twelve complete RM round-trips (three between each RM pair). After each partial bitstream load, a test vector suite was dispatched via CFU Playground:

- **example.v active:** `function_id=0, inputs_0=0x01020304 → result 0x0000000A` (byte-sum: 1+2+3+4). Verified across all trials.
- **donut.v active:** Fixed-point multiply vectors verified against software reference.
- **Cross-function isolation:** Issuing a `function_id` not supported by the loaded RM (e.g., requesting multiply while `example.v` is active) consistently returned `0x00000000` — the Decoupler’s safe-default output — with no pipeline stall or processor exception.

D. Decoupler Isolation Efficacy

Across all reconfiguration trials, no pipeline stall, processor exception, or UART session interruption was observed. The Decoupler’s `cmd_ready` hold-low behaviour during isolation was confirmed via ILA capture. The VexRiscv pipeline resumed CFU execution within one clock cycle of `decouple` de-assertion, with the first post-reconfiguration result matching the expected output of the newly loaded RM.

VI. DISCUSSION AND FUTURE WORK

A. Latency and Application Scheduling

A 63 ms JTAG reconfiguration window suits coarse-grained task schedulers where the CPU executes a software fallback during the swap interval. The PCAP hardware path (1.1 ms) approaches embedded RTOS context-switch overhead, making per-task CFU swapping feasible for multi-cycle accelerated workloads. The projected ICAP throughput (~50 MB/s from SPI flash) would place autonomous reconfiguration at approximately 10 ms for the bitstream sizes in this work — a significant improvement over JTAG.

B. Autonomous Reconfiguration Roadmap

The current implementation delivers partial bitstreams externally (JTAG host or PCAP via Linux). Two stages remain for full autonomy:

- 1) **Autonomous SPI Flash Fetch (near-term):** The ICAP controller FSM will be extended to read partial bitstreams from on-board QSPI flash at ~50 MB/s, eliminating host-side JTAG dependency and enabling standalone reconfiguration triggered by `pr_switch`. Estimated reconfiguration latency: ~10 ms for a 500 KB partial bitstream.
- 2) **DDR-Backed Bitstream Storage (longer-term):** Partial bitstreams stored in the Arty A7’s DDR3 (or Zynq DRAM on PYNQ-Z2) will be accessible to the ICAP controller via an AXI DMA path, enabling read throughputs of 400 MB/s and sub-millisecond reconfiguration latency for bitstreams 500 KB — approaching ZyCAP’s reported performance.

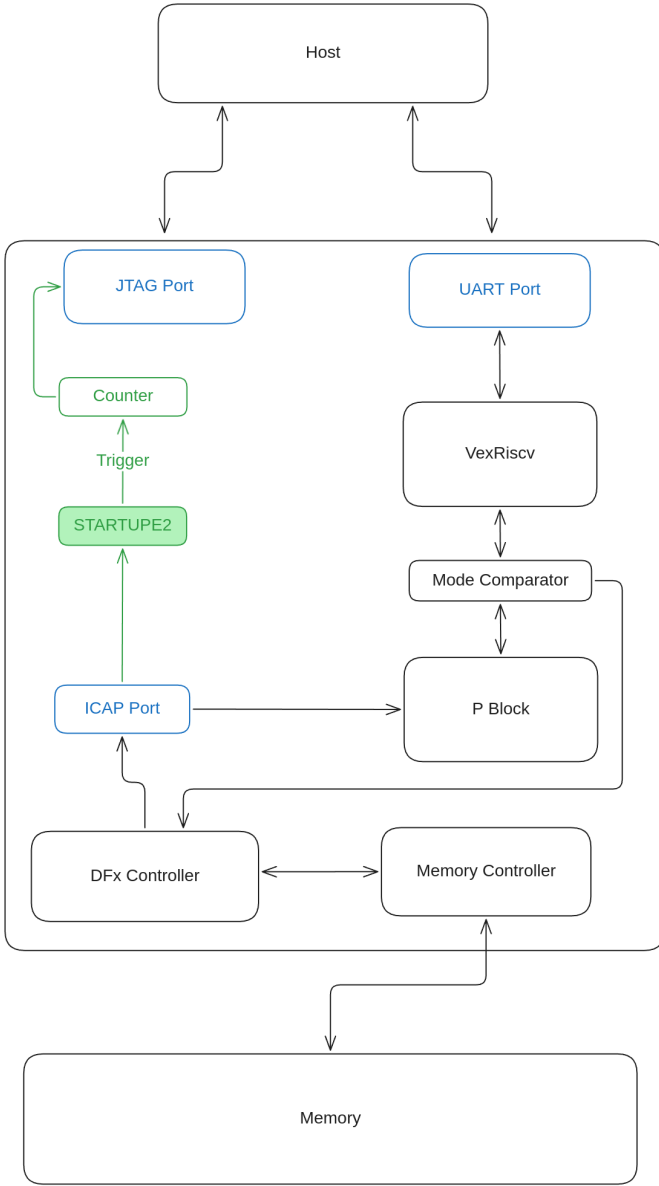


Fig. 9. Fig. 8: ICAP-based reconfiguration architecture — the target autonomous path where bitstream storage, retrieval, and delivery occur entirely within the FPGA fabric.

C. Multi-Slot CFU Extension

The current design provisions a single PRR for one active CFU. Extending to two or three independent PRRs, each addressable by a different `function_id` offset, would allow multiple concurrent accelerator personalities to be available to the VexRiscv pipeline — approaching the HPR multi-page model within the CFU Playground framework.

D. PCAP vs. ICAP Throughput Benchmarking

A controlled comparison of PCAP (PS-driven, DMA-backed, Zynq-7020) against ICAP (PL-driven, SPI-fetch, Arty A7) under identical partial bitstream sizes would produce empirical data directly informing the ZyCAP vs. pure-FPGA-controller design trade-off.

E. Graph-Based CFU Scheduling

At the firmware level, a runtime manager capable of profiling CFU call frequencies and predictively reconfiguring the PRR ahead of high-demand phases — analogous to DML’s task-graph scheduler — would convert the current manually triggered reconfiguration into a self-directed adaptive compute system.

VII. CONCLUSION

We have demonstrated the first integration of AMD’s DFX toolflow with Google’s CFU Playground framework, achieving runtime CFU swap on commodity 7-series hardware without processor reset. Hardware-timed reconfiguration latencies of approximately 63 ms (JTAG) and 1.1 ms (PCAP) were measured using a STARTUPE2-gated cycle counter, with the PCAP path delivering a 57× improvement. The DFX Decoupler provided reliable zero-output isolation across all validated events with zero pipeline stalls. Resource utilisation remains moderate (~19.6% LUT, ~7.8% FF), leaving substantial headroom for multi-partition architectures. The PR partition boundary and RM RTL were shown to be portable across 7-series device variants without modification. The gap between the current proof-of-concept and fully autonomous firmware-triggered reconfiguration is well-defined: all required primitives (ICAPE2, AXI Quad SPI, DFX Decoupler) are instantiated in the static bitstream, and the remaining work is principally an RTL integration exercise connecting the SPI flash read path to the ICAP data pipeline.

REFERENCES

- [1] AMD/Xilinx, “Vivado Design Suite User Guide: Dynamic Function eXchange,” UG909, 2023.
- [2] G. Clow *et al.*, “CFU Playground: Full-stack open-source framework for tiny machine learning (tinyML) FPGA co-design,” in *Proc. FCCM*, 2022.
- [3] T. Herpel *et al.*, “RV-CAP: RISC-V controlled adaptive partial reconfiguration,” in *Proc. FPL*, 2021.
- [4] M. Owaid *et al.*, “AMPER-X: Adaptive mixed-precision execution using dynamic partial reconfiguration,” in *Proc. DATE*, 2022.
- [5] H. Guo *et al.*, “RapidStream: Parallel physical implementation of FPGA HLS designs,” in *Proc. FPGA*, 2022.
- [6] D. Koch *et al.*, “ZyCAP: Efficient partial reconfiguration management on the Zynq SoC,” *IEEE Embedded Systems Letters*, vol. 5, no. 4, pp. 53–56, 2013.
- [7] M. Clark *et al.*, “HPR: Hierarchical partial reconfiguration for FPGA resource management,” in *Proc. FPL*, 2020.
- [8] K. Vipin and S. A. Fahmy, “FPGA dynamic and partial reconfiguration: A survey of architectures, methods, and applications,” *ACM Computing Surveys*, vol. 51, no. 4, 2018.
- [9] AMD/Xilinx, “7 Series FPGAs Configuration User Guide,” UG470, 2023.
- [10] A. Ferreira *et al.*, “DORA: Low-latency dynamic overlay for reconfiguration acceleration,” in *Proc. FPT*, 2024.